

Este boletín de ejercicios está orientado a practicar con

- ficheros
- programación orientada a objetos
- repaso general de todo el contenido

1. Crea una función en python que se llame `compararFicheros` y que reciba como argumento el nombre de dos ficheros de texto. La función debería de devolver un valor booleano indicando si el contenido de ambos ficheros es exactamente el mismo o no.

2. Tenemos un fichero llamado **estadisticas.txt**. El formato del fichero es el siguiente (pero el contenido puede variar, lógicamente):

```
Hombre
1.73
Mujer
1.68
Mujer
1.83
```

Realiza un programa que lea el contenido de ese fichero y muestre el número de hombres, el número de mujeres y la altura media (con dos decimales) de todos sin hacer distinción de sexo.

Por ejemplo, para el fichero del ejemplo anterior, la salida del programa sería esta:

```
Hombres: 1.
Mujeres: 2.
Estatura media: 1.75
```

El formato del fichero se supone correcto y comprobado y nunca va dar problemas

3. Un código bancario en formato IBAN en nuestro país está formado por 24 caracteres de los cuales los dos primeros son letras y los restantes 22 son dígitos numéricos comprendidos entre el 0 y el 9. Por ejemplo, el siguiente sería un IBAN válido

ES1234567890123456789012

Es corriente verlos escritos tanto de esta forma, sin espacios, como con diferentes separaciones para que sea más cómodo. Por ejemplo, los siguientes también serían IBAN válidos:

```
ES12 3456 7890 1234 5678 9012
ES12 3456 7890 12 3456789012
ES 12 3456 7890 123456789012
```

Escribe una función que reciba como argumento un nombre de fichero que tendrá un listado de códigos de este tipo. Algunos serán válidos y otros no. Tu programa debería de mostrar por consola sólo los válidos. Por ejemplo, si el contenido del fichero `codigos.txt` recibido fuese este:

```
ES1234567890123456789012
UK12345678901234567
3312 3456 7890 1234 5678 9012
ES12 3456 7890 12 345678901299999
FR32 5456 7898 22 0456789012
ES 12 3456 7XX0 123456789012
```

Tu programa debería de dar la siguiente salida en consola:

Códigos correctos en el fichero `codigos.txt`:

País: ES
DC: 12
Entidad: 3456
Sucursal: 7890
DC cuenta: 12
Número de cuenta: 3456789012

País: FR
DC: 32
Entidad: 5456
Sucursal: 7898
DC cuenta: 22
Número de cuenta: 0456789012

Hay 2 códigos correctos y 4 incorrectos

Como puedes ver, los distintos elementos del código son los dígitos del mismo tal y como aparecen ordenados en él. Y, como siempre, cuida de que la salida sea igual a la que se muestra aquí

4. Escribe un programa usando POO que, tomando el mismo fichero `codigos.txt` del ejercicio 3, tenga una clase que se llame IBAN donde guarde la información de los códigos IBAN correctos que se hayan leído del fichero.

Tu clase debería de tener, al menos, un constructor para crear el objeto y un método llamado `mostrar` para que se visualice la información del código por consola. El constructor recibirá el código IBAN en formato de cadena de texto. Así:

```
codigoIBAN01 = Iban("ES1234567890123456789012")
```

Y debería de admitir como argumento de entrada cualquier IBAN válido independientemente de los

espacios en blanco tal y como se describe en el anterior ejercicio.

Tu clase deberá de contar con atributos separados para los diferentes elementos del IBAN (pais, dc, entidad, sucursal, dc_cuenta y num_cuenta). El método mostrar, antes mencionado, nos listará los códigos de forma similar a como se describe en el ejercicio anterior.

En definitiva, tu programa debe de funcionar igual que el ejercicio 3 pero usando POO. Puedes, si quieres, hacer este ejercicio en lugar del 3 (y tendrás la misma puntuación en ambos) pero no al revés.

5. En los centros de datos es normal anonimizar de alguna forma los ficheros de clientes para poder hacer pruebas con ellos sin vulnerar las leyes de protección de datos. Una de las formas mas utilizadas es “barajar” los datos de los clientes originales. Imagina que el fichero de clientes de tu empresa es el siguiente:

Diego Norrea 28222777J
Inés Perado 07333888X
Demetrio Imedio 97221345Y
Roberto Rija 22876345M
Rubén Tosidad 12987543C
Armando Adistancia 32879563V
Germán Tequilla 18000777H

Cuenta con que la estructura del fichero es siempre correcta y todas las líneas están formadas por la misma estructura: nombre, primer apellido y DNI separados por un único espacio en blanco

Queremos hacer una función que reciba como argumento un nombre de fichero. Tu programa debería de leer de un fichero llamado clientes.txt con una estructura como la anterior y escribir en el fichero que has recibido como argumento los datos “barajados”. Por ejemplo así:

Armando Imedio 12987543C
Imedio Norrea 28222777J
Roberto Adistancia 97221345Y
Inés Rija 18000777H
Diego Perado 22876345M
Rubén Tequilla 07333888X
Germán Tosidad 32879563V

Como puedes observar, lo que hacemos es mezclar de forma aleatoria los nombres, los apellidos y los dnis de los clientes a la hora de generar el fichero resultante.

6. Escribe un programa usando POO que, tomando el mismo fichero clientes.txt del ejercicio anterior, tenga una clase que se llame cliente donde guarde la información de los clientes que se hayan leído del fichero.

Tu clase debería de tener un constructor para crear el objeto que reciba la línea tal y como se lee del fichero. Así:

```
cliente01 = Cliente("Diego Norrea 28222777J")
```

Tu clase deberá de contar con atributos separados para el nombre, el apellido y el NIF.

Debes de crear un método que se llame mostrar que nos muestre la información del cliente por consola en el siguiente formato:

```
28222777J - Norrea, Diego
```

Por último, usando esta clase como soporte, haz un listado del contenido del fichero por consola que debería de quedar así:

```
28222777J - Norrea, Diego  
07333888X - Perado, Inés  
97221345Y - Imedio , Demetrio  
22876345M - Rija, Roberto  
12987543C - Tosidad, Rubém  
32879563V - Adistancia, Armando  
18000777H - Tequilla, Germán
```

7. Te ha contratado la policía nacional para que hagas un programa que permita ver si un ciudadano tiene ficha por delitos previos y mostrar los resultados. El archivo de la policía se llama delincuentes.txt y tiene este formato:

```
- Diego Norrea, 35  
Robo con violencia  
- Demetrio Imedio, 53  
Acoso laboral  
Evasión de impuestos  
Corrupción  
- Inés Perado, 48  
Hurto  
Extorsión
```

Como ves, los nombres de los delincuentes siempre empiezan por un guión y un espacio en blanco y terminan con una coma y su edad. A continuación aparecen los delitos cometidos uno por línea

Tu programa debería de pedir por teclado el nombre de un sospechoso y decir si tiene o no antecedentes. Por ejemplo, en el caso de alguien sin antecedentes:

```
Introduce el nombre del ciudadano: Ricardo Borriquero
```

Sin antecedentes penales

En el caso de tener antecedentes debería de listarlos de la siguiente forma:

Introduce el nombre del ciudadano: Ines Perado

Edad: 48 años

Antecedentes penales:

Hurto

Extorsión

NOTA: Piensa que no sabes cuantos antecedentes puede tener el individuo pero que si está en el fichero debe de tener al menos uno

8. Estamos generando un fichero con los nombres de los personajes para un juego de rol y queremos que estos sean aleatorios. Para ello contamos con dos listas, una de nombres y otra de apellidos como estas:

**nombres=["Ash", "Momo", "Monkey", "Naruto", "Nico", "Ken",
"Roronoa", "Touka"]**

**apellidos=["Ketchum", "Ayase", "D. Luffy", "Uzumaki", "Robin",
"Kaneki", "Zoro", "Kirishima"]**

Hay, como ves, ocho nombres y ocho apellidos. Es el máximo de personajes que admitirá nuestro juego. Tu programa debería de pedir por teclado un número entre el 1 y el 8 que serían el número de personajes que necesitamos para la partida y generar los nombres tomando aleatoriamente un nombre y un apellido sin repetir nunca ninguno. Por ejemplo así:

Cuantos personajes tendrá tu partida: 4

Personajes:

Momo Uzumaki

Touka Kirishima

Ash Robin

Ken Ayase

Además de generar la salida anterior tu programa debería de crear un fichero que se llame personajes.txt y escribir en él el nombre de los personajes generados. En el ejemplo anterior tu fichero personajes.txt tendría este contenido:

Momo Uzumaki

Touka Kirishima

Ash Robin

Ken Ayase

Tu programa debería de detectar que en la entrada por teclado no se meta un número entero o que este sea inferior a 1 o superior a 8. En todos esos casos que le impedirían funcionar correctamente debería de mostrar un error en consola y no hacer nada mas.

9. Tenemos un fichero llamado alumnos.txt con el siguiente formato:

```
Pedo, Aitor: 5, 7  
Casitas, Armando: 4.5, 9  
Lega, Francisco: 8.5, 6.5, 6.3  
Ligro, Penélope: 1.1
```

Cada alumno aparece seguido de una serie de notas (números decimales entre 1 y 10)

Haz un programa en python que lea de ese fichero y nos de una salida en consola listando sólo los alumnos que tienen todas las calificaciones aprobadas (notas superiores o iguales a 5) y la media aritmética:

```
Aitor Pedo - 6  
Francisco Lega - 7.1
```

Donde el número que aparece es la media aritmética de las notas de cada alumno

NOTAS:

- El contenido del fichero alumnos.txt que aparece aquí es sólo un ejemplo. Tu programa debería de funcionar correctamente con cualquier fichero con ese formato.
- En el fichero, la separación entre el nombre y la nota es de un signo de dos puntos seguido de un espacio. La separación entre cada dos notas es una coma y un espacio.
- Cada alumno tiene una nota como mínimo, pero el número es variable, indeterminado y puede ser diferente por cada alumno
- Las medias se redondearán a un decimal máximo
- Suponemos que el formato del fichero es siempre correcto y no tiene errores

10. Sabrás, por el módulo de Redes, que las redes de Clase A tienen máscara /8, las clase B máscara /16 y las clase C /24. Tenemos un fichero llamado redes.txt con el siguiente formato:

```
192.168.10.0/24  
200.30.30.0/24  
172.16.0.0/16  
11.0.0.0/8  
130.20.0.0/16
```

Haz un programa en Python que lea de ese fichero y nos de una salida en consola listando las redes clasificadas por tipo de la siguiente forma:

Redes Clase A:
11.0.0.0/8

Redes Clase B:
172.16.0.0/16
130.20.0.0/16

Redes Clase C:
192.168.10.0/24
200.30.30.0/24

NOTAS:

- El contenido del fichero redes.txt que aparece aquí es sólo un ejemplo. Tu programa debería de funcionar correctamente con cualquier fichero con ese formato.

11. Tenemos un fichero cuyo formato es el siguiente (pero el contenido puede variar, lógicamente):

1234.5
725.3
pepe
4.37
12
ani33kk
1285.3

Realiza una función que reciba como argumento el nombre del fichero. Tu función debe de leer el contenido y mostrarnos una salida como esta (para el anterior fichero):

Número de datos válidos: 5
Número de datos inválidos: 2
Mínimo: 4.37
Máximo: 1285.3
Media aritmética: 652.294

Tu función debería de detectar las líneas incorrectas y no tenerlas en cuenta pero informar de ellas en la salida tal y como ves en el ejemplo anterior. Se considerará incorrecta cualquier línea que no pueda convertirse a numérica con o sin decimales

La media aritmética debe de expresarse con tres decimales máximo

No olvides tener en cuenta las excepciones que puedan ocasionarse por el trabajo con el fichero

12. Queremos hacer un programa en python que sirva para evaluar los resultados de un test. El fichero manejará dos ficheros: uno llamado soluciones.txt y otro llamado respuestas.txt

El fichero llamado soluciones.txt tiene esta estructura:

A, C, C, D, B, A, D, A, B, A

Como ves, son los resultados de un examen de 10 preguntas tipo test. Las opciones válidas son A, B, C o D y ninguna otra y deben de aparecer como aquí arriba: en mayúsculas y separadas por un espacio y una coma. Si el fichero de soluciones no es correcto de dará un mensaje en consola y no se hará nada mas.

El fichero llamado respuestas.txt tendrá la siguiente estructura:

Claudia Pasón: A, C, C, D, A, A, C, A, B, A

Germán Tequilla C, B, C, A, B, D, C, A B

Alfonso Litario: A, B, C, D, B, A, D, A, B, A

Como ves, hay una linea por cada examinado. Despues del nombre hay dos puntos y un espacio y por último las respuestas al test también separadas por coma y espacio. Si alguna línea no es correcta no se tendrá en cuenta a la hora de procesar el fichero y se ignorará, pero si se verán las demás. En el anterior ejemplo la línea 2, por ejemplo, es errónea

Tu programa debería de mostrar las notas de las líneas correctas de esta forma:

Claudia Pasón: 7.6

Alfonso Litario: 8.8

Para obtener la nota de cada examinado se suma 1 por cada respuesta correcta y se resta 0,2 por cada respuesta incorrecta. La nota se expresará con dos decimales máximo

13. Escribe un programa en python para desarrollar una función de login con las siguientes características:

- Los usuarios y contraseñas válidos estarán almacenados en un fichero con la sintaxis usuario:contraseña. Suponemos que el fichero es correcto y no habrá errores en su formato. Tampoco puede haber usuarios repetidos. Un ejemplo de fichero podría ser este:

```
josemaria:abc  
sara:romeo1  
alberto:M4d4g4scar+  
juan:TOPO
```

- Los dos puntos que separan usuario y contraseña son eso, un separador y no pertenece ni a uno ni a otro. Por motivos evidentes el carácter : no se permite que forme parte ni del usuario ni de la contraseña. Es decir, en cada línea del fichero debe de aparecer un carácter : como separador pero no puede aparecer ningún otro.

- La ubicación del fichero de contraseñas se guarda en una variable del programa llamada

fichero. Así:

fichero = “/home/josemaria/login.txt”

Tu programa debería de pedir por teclado usuario y contraseña y validar si son correctas contrastando con lo guardado en el fichero. Si el login es correcto debería de decirlo. Si no lo es debería de informar del problema encontrado, reconociendo al menos las siguientes casuísticas:

- **fichero inexistente o imposible acceder a él**
- **fichero vacío**
- **usuario no encontrado**
- **contraseña incorrecta**

14. Haz un programa en Python que permita añadir elementos en el fichero del ejercicio anterior. Tú programa debería de pedirte un usuario y una contraseña por consola y grabar la pareja en el fichero con la misma sintaxis indicada. Para mayor seguridad, debería de pedirte la contraseña dos veces y asegurarse de que ambas son iguales. Si no, no debería de hacer la grabación y debería de mostrar un mensaje de error por pantalla. Un ejemplo de ejecución podría ser así:

Introduce el nombre del usuario: manolo
Introduce la contraseña: 123
Vuelve a introducir la contraseña de nuevo: 1234
Las contraseñas no son iguales. No se puede grabar la nueva cuenta

Otro ejemplo:

Introduce el nombre del usuario: manolo
Introduce la contraseña: 123
Vuelve a introducir la contraseña de nuevo: 123
Cuenta de usuario grabada correctamente

15. Haz un programa en Python que convierta las parejas usuario:password en objetos de una clase Consideraciones a tener en cuenta:

- El constructor de la clase debe de recibir como argumento una línea con el usuario y la contraseña con el mismo formato que en el ejercicio 1 (“josemaria:abc123”, por ejemplo). No tiene que leerla del fichero ni del teclado. Por lo demás, define la clase de la forma mas apropiada que veas para lo que tienes que hacer.
- Crea un método que nos de una valoración de la solidez de la contraseña teniendo en cuenta su longitud y los tipos de caracteres que usa con la siguiente pauta. Tú método dará una

puntuación de la contraseña en función de lo siguiente:

Contraseña de mas de 8 caracteres +1
Contraseña que incluye letras (ya sean mayúsculas o minúsculas) +1
Contraseña que incluye mayúsculas y minúsculas simultaneamente +1
Contraseña que incluye números +1
Contraseña que incluye otros signos + 1

La salida por consola de este método debería de ser así:

Usuario: josemaria
Password: abc123
Fortaleza de la contraseña: 2

Otro ejemplo:

Usuario: alberto
Password: M4d4g4scar
Fortaleza de la contraseña: 4

16. Haz un programa en Python que haga lo siguiente:

- Lea del fichero del ejercicio 11 y convierta las parejas usuario:contraseña en objetos de una clase
- Grabe los objetos en un fichero binario que se llame login.bin
- Lea del fichero binario que has escrito y muestre el contenido de los objetos por consola

Tú programa debería de funcionar independientemente del número de elementos que haya en el fichero, tanto a la hora de grabarlo en disco como de leerlo posteriormente.

Ejemplo de ejecución:

Fichero origen: /home/josemaria/login.txt
Fichero destino: login.bin
Número de cuentas encontradas: 2
Listado de cuentas:

Usuario: josemaria
Password: abc123
Fortaleza de la contraseña: 2

Usuario: alberto
Password: M4d4g4scar
Fortaleza de la contraseña: 4

17. Queremos hacer un programa en Python que revise que la sintaxis de un archivo es correcta. El fichero debería de tener una estructura como la que sigue:

Imedio, Demetrio;Programador Categoría 2;1599.56
Borriquero, Luis Ricardo;Analista;1341.60
Lorin, Francisco;Administrativo;1095
Cortada del Rosal, Rosa;Administrador de bases de datos;2256.99

- Los elementos están separados por punto y coma.
- El primer elemento son los apellidos (una o varias palabras y sólo puede haber letras) separados del nombre (también una o varias palabras y sólo puede haber letras) por una coma y un espacio
- El segundo elemento el puesto de trabajo (una o varias palabras también pero en este caso puede haber números)
- El tercer y último campo es el salario mensual (un número que puede tener decimales o no)
- Cualquier línea que no cumpla con lo listado anteriormente se considerará errónea

Tú programa debería de leer de un fichero origen y escribir en un fichero destino sólo las líneas que sean correctas. Las que no sean correctas deberían de listarse por pantalla.

Los ficheros de origen y de destino deberían de aparecer como variables del programa para que sea mas fácil la corrección. Así, por ejemplo:

origen = /home/josemaria/origen.txt
destino = /home/josemaria/salida.txt

18. Queremos añadir al fichero del ejercicio anterior un cuarto campo que sea la edad del trabajador. Tu programa deberá de mostrar el nombre de cada persona del fichero pero no como aparece en él, sino poniendo antes el nombre y a continuación los apellidos) y preguntarnos la edad que introduciremos por teclado. Un ejemplo de ejecución partiendo del fichero anterior sería como sigue:

Demetrio Imedio. ¿Cuál es su edad? 34
Luis Ricardo Borriquero. ¿Cuál es su edad? 46
Francisco Lorin. ¿Cuál es su edad? 24
Rosa Cortada del Rosal. ¿Cuál es su edad? 58

En este caso no habrá que hacer comprobaciones previas del formato del fichero que se supone correcto, pero si de que lo que se introduce por teclado es un número entero superior o igual a 18 e inferior a 67 (se trata de un fichero de trabajadores y hay que ser mayor de edad y no haber cumplido la edad de jubilación ordinaria).

El fichero, una vez añadidos los datos, debería de quedar así:

Imedio, Demetrio;Programador Categoría 2;1599.56;34
Borriquero, Luis Ricardo;Analista;1341.60;46
Lorin, Francisco;Administrativo;1095;24
Cortada del Rosal, Rosa;Administrador de bases de datos;2256.99;58

En este caso el fichero final debe de llamarse igual que el original, así que sólo habrá una variable con el nombre del único fichero con el que puedes trabajar. Así, por ejemplo

fichero = /home/josemaria/empleados.txt

19. Vamos a hacer ahora a crear un programa con una clase que se llame Empleado y que contenga los siguientes atributos:

- nombre
- apellidos
- cargo
- salario
- edad

Tú clase recibirá en el constructor una línea como la escrita en el fichero anterior. Así, por ejemplo:

Empleado("Imedio, Demetrio;Programador Categoría 2;1599.56;34")

Deberás, además, crear una función llamada mostrar que muestre la siguiente salida:

Empleado: Demetrio Imedio
Cargo: Programador Categoría 2
Años hasta su jubilación ordinaria: 33
Salario neto anual: 22393.84

Sabiendo que la jubilación ordinaria se produce a los 67 años y que el salario neto anual se calcula multiplicando el salario mensual por 14 pagas que recibe el empleado al año

20. Crea una función que reciba un empleado (de la clase que has creado en el ejercicio anterior) y el nombre de un fichero y te añada el empleado al fichero. La llamada a esta función debería de ser así:

grabarEmpleado("/home/josemaria/empleados.bin", empleado1)

Siendo /home/josemaria/empleados.bin el fichero donde haremos la grabación y empleado1 el nombre del objeto de la clase Empleado que queremos grabar en el.

Las condiciones que debes de cumplir son las siguientes:

- El fichero debe de ser binario porque queremos grabar los empleados como objetos
- Puede que el fichero no exista, que exista y esté vacío o que ya tenga empleados grabados en él. Tú programa debería de ser capaz de resolver cualquiera de esas tres situaciones: Si no existe, lo creas. Si existe y está vacío escribes en él tu objeto. Si el fichero ya tiene objetos grabados el nuevo debería de añadirse a los que ya existen.

Tu programa debería, finalmente, volver a abrir el fichero y hacer un listado del nombre, los apellidos y la edad de todos los empleados que existan grabados en él con este formato:

Demetrio Imedio (34)
Luis Ricardo Borriquero (46)
Francisco Lorín (24)
Rosa Cortada del Rosal (58)

21. Tenemos un fichero con un formato como el que sigue:

1 Bulbasaur; 2 Ivysaur; 3 Venasaur
25 Pikachu; 26 Raichu
122 Mr. Mime
227 Skarmory

Dónde cada línea del fichero se corresponde con una línea evolutiva de un Pokemon de la primera generación. Tu fichero deberá de comprobar lo siguiente:

- En cada línea aparece al menos un pokemon y como máximo tres
- Por cada pokemon aparece su número de la pokedex y su nombre. El separador entre pokemons es siempre el símbolo ; y un espacio en blanco
- El número de la pokedex es un número positivo mayor o igual a 1 e inferior o igual a 151
- El nombre del pokemon puede contener espacios y el punto (.) pero nunca números u otros símbolos no alfabéticos
- Tu programa debe de tener dos variables claramente definidas con los nombres de los ficheros de entrada y de salida. Así por ejemplo:

entrada = “/home/josemaria/pokemon_in.txt”
salida = “/home/josemaria/pokemon_out.txt”

- Cuando tu programa detecte una línea errónea debe de mostrarla por consola y decir porque la considera errónea
- Al final, tu programa debe decir cuantos pokemons (pokemons, no líneas) correctos hay en el fichero y escribir sólo las líneas que considera correctas en el fichero de salida. Una salida de ejemplo podría ser así:

Línea errónea detectada: 227 Skarmory
El número de la pokedex no es de la primera generación

Se han grabado 6 pokemons correctos en el fichero de salida

22. Partimos ahora de un fichero como el del ejercicio anterior pero ya validado y correcto donde no tienes que comprobar que haya líneas erróneas porque es seguro que no las hay. Puedes hacer las pruebas con el siguiente:

**1 Bulbasaur; 2 Ivysaur; 3 Venasaur
25 Pikachu; 26 Raichu
122 Mr. Mime**

Tu programa pedirá por teclado un pokemon (el usuario puede escribir indistintamente tanto un número de la pokedex como el nombre del pokemon) y el programa mostrará por pantalla sus datos de la siguiente forma:

**Introduce el pokemon en el que estás interesado: Pikachu
Pikachu. Su número de la Pokedex es el 25
No evoluciona de ningún otro. Evoluciona en Raichu (26)**

**Introduce otro pokemon o escribe Exit para salir: José María
Ese Pokemon no está registrado en la pokedex**

**Introduce otro pokemon o escribe Exit para salir: 2
Iviysaur. Su número de la Pokedex es el 2
Evoluciona de Bulbasaur(1). Evoluciona en Venasaur(3)**

**Introduce otro pokemon o escribe Exit para salir: 45,66AXX
Eso no parece corresponder a un nombre de pokemon o código de la pokedex**

**Introduce otro pokemon o escribe Exit para salir: Mr. Mime
Mr. Mime. Su número de la Pokedex es el 122
No evoluciona de ningún otro. No evoluciona a ningún otro**

**Introduce otro pokemon o escribe Exit para salir: Exit
Gracias por consultar la pokedex**

Lógicamente, una entrada invalida será detectada por el programa e informará de ello por consola como se puede ver aquí arriba

NOTA IMPORTANTE: Si, sabemos que Pikachu evoluciona de Pichu, pero Pichu no es un pokemon de la primera generación que son los únicos que conoce nuestra pokedex

23. Tenemos necesidad de mejorar nuestra pokedex y vamos a utilizar para ello programación orientada a objetos. Además de los datos que ya manejamos (número de la pokedex, nombre del pokemon, pokemon del que evoluciona y pokemon a que evoluciona) queremos guardar el tipo o tipos del Pokemon. Un ejemplo de como debería de funcionar nuestro programa es este:

```
pokemon1 = Pokemon(10, "Caterpie")  
pokemon2 = Pokemon(11, "Metapod")  
pokemon3 = Pokemon(12, "Butterfree")  
  
pokemon1.evoluciona(pokemon2)  
pokemon2.evoluciona(pokemon3)
```

```
pokemon1.tipo("Bicho")
pokemon2.tipo("Bicho")
pokemon3.tipo("Bicho")
pokemon3.tipo("Volador")
pokemon1.mostrar()
pokemon2.mostrar()
pokemon3.mostrar()
```

Y la salida por consola de las tres llamadas al método mostrar serían así:

10. Caterpie

- No se conoce de donde evoluciona
- Evoluciona en Metapod
- Tipos: Bicho

11. Metapod

- Evoluciona de Caterpie
- Evoluciona en Butterfree
- Tipos: Bicho

12. Butterfree

- Evoluciona de Metapod
- No se conoce a donde evoluciona
- Tipos: Bicho, Volador

- Fíjate que un pokemon puede pertenecer a uno o mas tipos y tu clase debe de contemplarlo
- En total, además del constructor debes de crear tres métodos: evoluciona, tipo y mostrar
- El método evoluciona, por si no queda claro, funciona de la siguiente forma: cuando se ejecuta `pokemon1.evoluciona(pokemon2)` el método debe de registrar que el `pokemon1` evoluciona al `pokemon2` pero también que el `pokemon2` evoluciona del `pokemon1`.

24. Queremos implementar un método que nos permita jugar eligiendo tres pokemons de forma aleatoria de entre los guardados en un fichero.

NOTA IMPORTANTE: Puedes partir de la clase que has creado en el ejercicio anterior o si quieres, para simplificar o porque no lo has hecho, puedes usar una mas simple sólo con el constructor. No te va a hacer falta nada mas

Lo primero que tiene que hacer tu programa es crear los pokemons y guardarlos en un fichero binario:

```
fichero = "/home/josemaria/opciones.bin"
```

```
p1 = Pokemon(10, "Caterpie")
p2 = Pokemon(11, "Metapod")
p3 = Pokemon(12, "Butterfree")
p4 = Pokemon(25, "Pikachu");
```

```
p5 = Pokemon(26, "Riachu");
```

```
grabarFichero(fichero, p1, p2)
```

```
grabarFichero(fichero, p3, p4, p5)
```

La función grabarFichero que tienes que implementar puede recibir cualquier número de argumentos (uno, dos, tres, etc.) y añadirá los objetos a los ya existentes en el fichero o creará un fichero nuevo si este no existe o está vacío

Además, necesitamos implementar una función que elegirá los pokemons con los que combatirá cada jugador. Esta función se invocará así:

```
combate(fichero)
```

Y su salida por consola será esta:

```
El jugador 1 combate con Metapod y Riachu
```

```
El jugador 2 combate con Caterpie y Pikachu
```

La función combate elige dos pokemons para cada jugador de forma aleatoria de entre todos los que aparezcan grabados en el fichero de forma que no pueden aparecer pokemons repetidos. Si el fichero no existe, el contenido no es válido o no tiene grabados al menos 4 pokemons tu programa debería de mostrar un error.